MÓDULO: SISTEMAS DE GESTIÓN EMPRESARIAL (SGE - 2º DAM)

# POO Idiomática en Python

## Sesión 2: Clases de Datos, Encapsulación Real y Métodos Mágicos

> *“Del modelo anémico de Java a entidades ricas de negocio en Python 3.10+”*

1. El Antipatrón:
Código Ceremonial
2. El Nuevo Estándar:
@dataclass
3. Seguridad:
Inmutabilidad
(Value Objects)
4. Control:
Validación con
__post_init__
5. Poder Nativo:
Dunder Methods
6. Profesionalismo:
Encapsulación
Pragmática
7. El Desafío:
Reto Práctico
Objetivo: 60 minutos para construir una entidad de negocio completa y segura sin escribir una sola línea inútil.
Gemini Notebook

# El Peso Ceremonial (Java)

```java
public class LineaPedido {
    private String sku;
    private int unidades;
    private double precio;

    public LineaPedido() {}

    public LineaPedido(String sku, int unidades,  double precio) {
        this.sku = sku;
        this.unidades = unidades;
        this.precio = precio;
    }

    public String getSku() { return sku; }
    public void setSku(String sku) { this.sku = sku; }
    public int getUnidades() { return unidades; }
    public void setUnidades(int unidades) { this.unidades = unidades; }
    public double getPrecio() { return precio; }
    public void setPrecio(double precio) { this.precio = precio; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        LineaPedido that = (LineaPedido) o;
        return unidades == that.unidades &&
             Double.compare(that.precio, precio) == 0 &&
             java.util.Objects.equals(sku, that.sku);
    }

    @Override
    public int hashCode() {
        return java.util.Objects.hash(sku, unidades, precio);
    }

    // More methods, getters, setters, toString...
}
```

# Ingeniería Declarativa (Python)

```python
from dataclasses import dataclass

@dataclass
class LineaPedido:
    sku: str
    unidades: int
    precio: float
```

Gemini Notebook

**Sintetiza __init__:** Construye la inicialización basada en el orden de los campos.
from dataclasses import dataclass
@dataclass
class Cliente:
cif: str
razon_social: str
activo: bool = True
**Genera __repr__:** Crea una representación textual automática y legible para logs.
**Provee __eq__:** Implementa comparación lógica campo a campo por defecto.
Nota: Los valores por defecto (ej. activo: bool = True) siempre van al final.
Gemini Notebook

```python
@dataclass(frozen=True)
class Dinero:
    importe: float
    divisa: str = 'EUR'

precio_tarifa = Dinero(120.50)
precio_tarifa.importe = 90.0
```

```
FrozenInstanceError:
cannot assign to field 'importe'
```

**Contexto SGE:** Ideal para Value Objects sin identidad conceptual (monedas, tipos impositivos, rangos de fechas). Dos billetes de 100€ son iguales por su valor, no por su número de serie.

Gemini Notebook

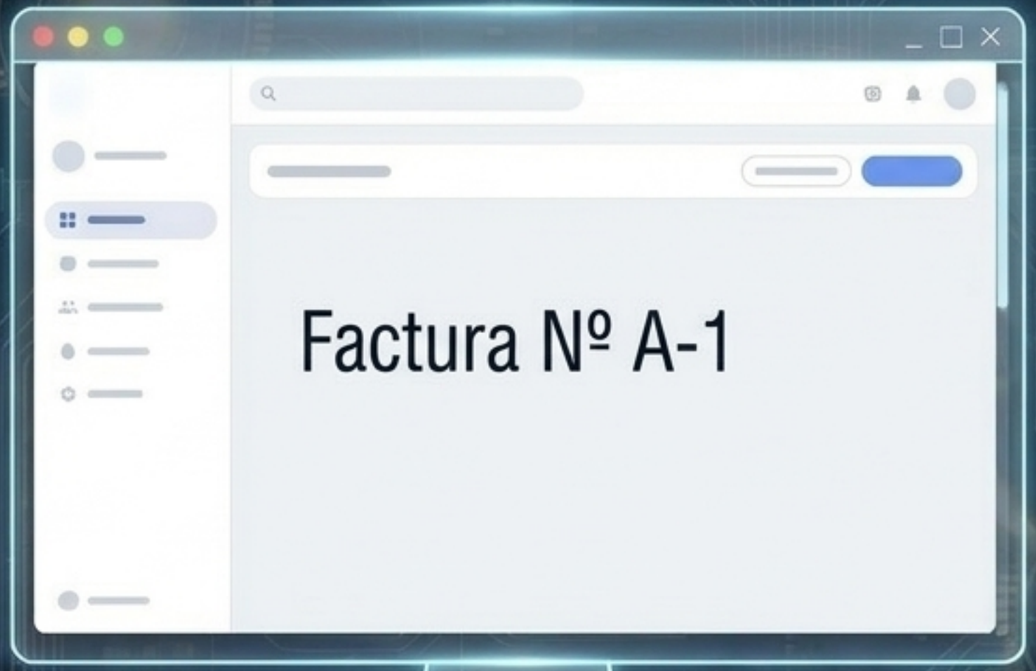
Gemini Notebook

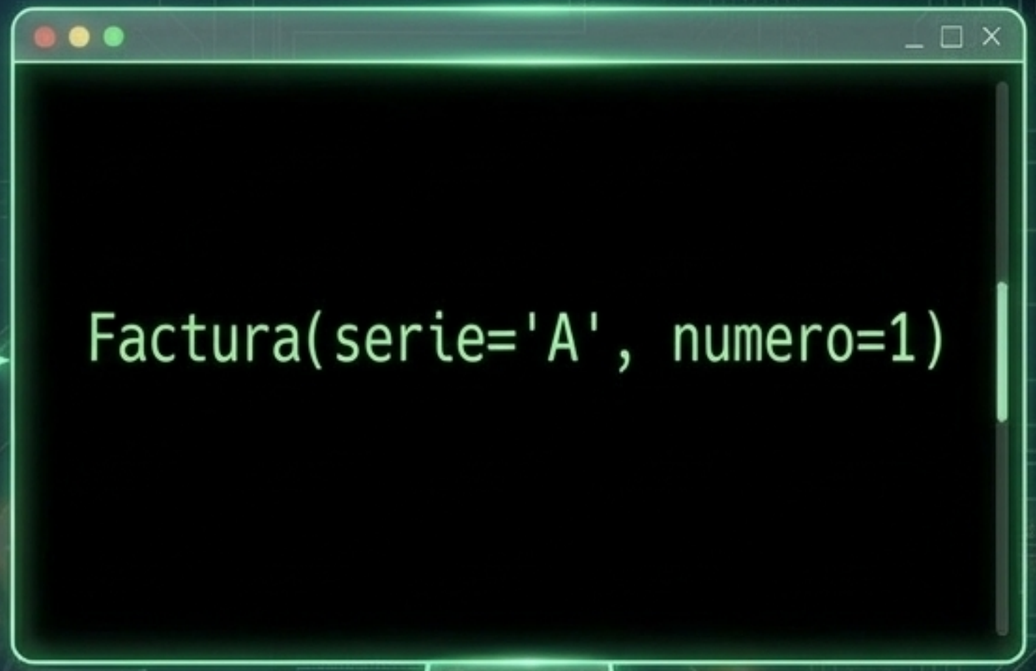
# El Protocolo del Modelo de Datos (Dunder Methods)

**Concepto clave: "Double Underscore"** = Conectar con el motor interno de Python.

Gemini Notebook

# Representación Dual del Objeto: __str__ vs __repr__




**Para el Cliente (__str__)**

```
def __str__(self):
    return f'Factura Nº {self.serie}-{self.numero}'

def __repr__(self):
    return f'Factura(serie={self.serie}{self.numero})'
```

**Para el Desarrollador (__repr__)**

**Concepto clave:** Representaciones duales para distintos propósitos (legibilidad vs precisión técnica).

Gemini Notebook

# Igualdad Lógica vs Identidad Física

**Contenido (==)**: Las cajas contienen los mismos items. Python evalúa a True (invoca __eq__). IGUALDAD LÓGICA.

**Ubicación (is)**: Las cajas están en estantes diferentes. Python evalúa a False. IDENTIDAD FÍSICA EN MEMORIA.

**Regla de Oro**: Usar 'is' exclusivamente con valores del sistema (if var is None). Usar '==' para todo el negocio.

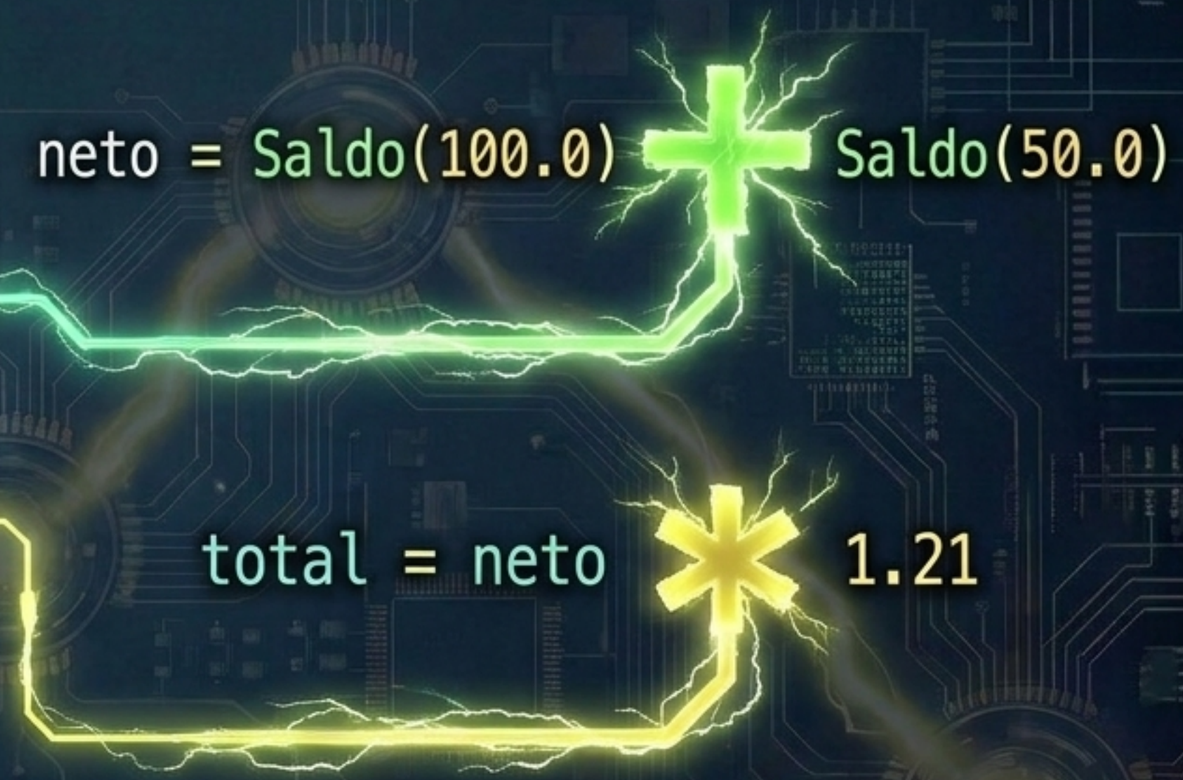
Gemini Notebook

# Sobrecarga de Operadores Aritméticos

```python
@dataclass(frozen=True)
class Saldo:
    cantidad: float

    def __add__(self, otro):
        return Saldo(self.cantidad +
otro.cantidad)

    def __mul__(self, factor):
        return Saldo(self.cantidad *
factor)
```




La suma no muta el original, da a luz a un nuevo Saldo (Inmutabilidad).

Gemini Notebook

# Emulando Tipos Nativos

## El problema en Java

```
remesa.getFacturas().size()
```

Expone la lista interna al consumidor.

## La elegancia Python

```
len(remesa)
```

**Integración total**: Al implementar __len__ e __getitem__, la clase se comporta mágicamente como una lista nativa (soporta bucles for y corchetes).

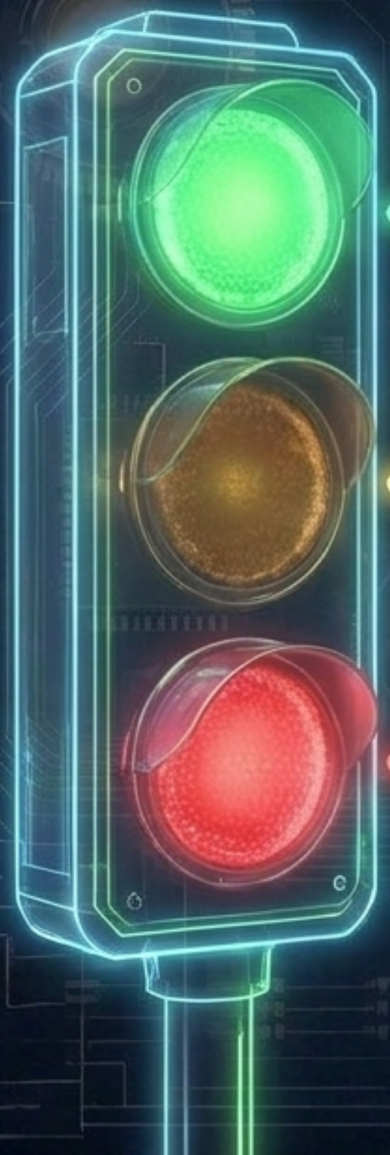
```python
class RemesaCobro:
    def __init__(self, facturas):
        self._facturas = facturas

    def __len__(self):
        return len(self._facturas)

    def __getitem__(self, pos):
        return self._facturas[pos]
```

Gemini Notebook

# Convenciones y Visibilidad



**Público** `self.total`

Accesible y modificable libremente.

**Protegido** `self._total`

Convención comunitaria:
"No lo toques desde fuera, es interno".

**Privado Formal** `self.__total`

Aplica Name Mangling (el intérprete lo renombra internamente para evitar colisiones).

*"Todos somos adultos aquí"*

Python confía en el programador frente a la sobre-restricción del compilador.

Gemini Notebook

# Desterrando Getters (@property)

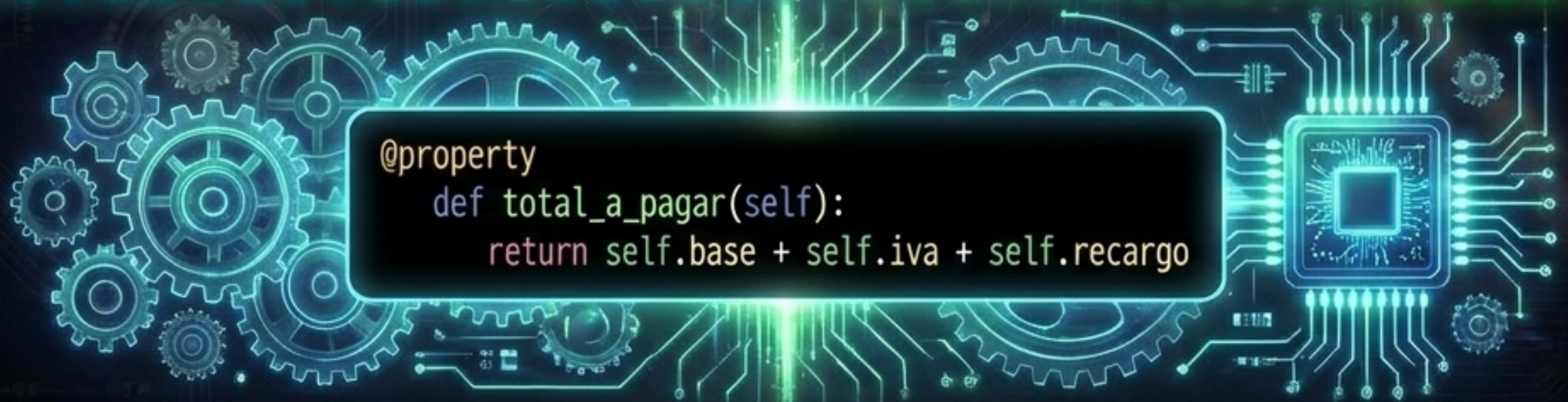
`objeto.getTotalA_Pagar()`

Sintaxis pesada, llamadas con paréntesis.

`objeto.total_a_pagar`

Lectura limpia como atributo.

`objeto.total_a_pagar`



```python
@property
    def total_a_pagar(self):
        return self.base + self.iva + self.recargo
```

La interfaz es un atributo estático, el interior es un método dinámico.

Gemini Notebook

# Asignación Controlada (@setter)

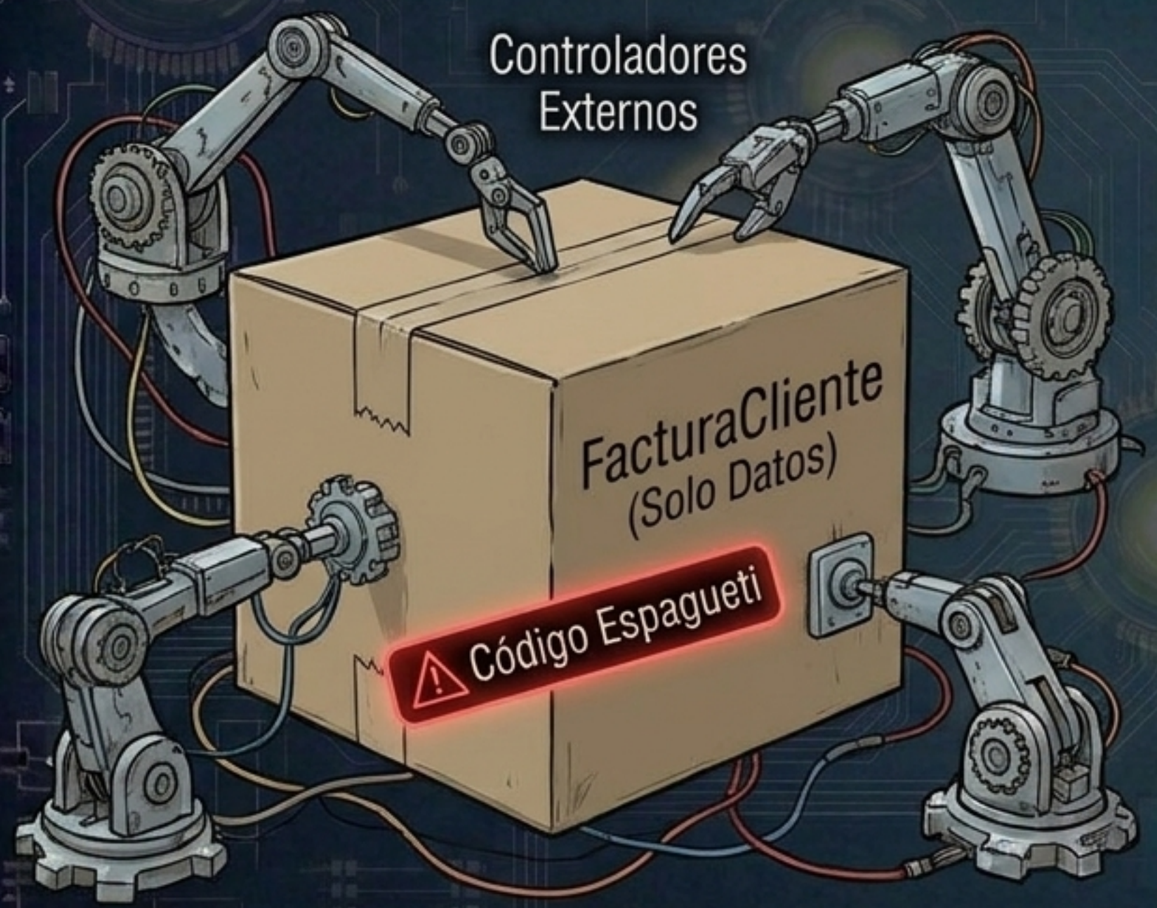
```python
@precio.setter
def precio(self, valor):
    if valor < 0:
        raise ValueError('Precio negativo')
    self._precio = valor
```

La validación activa ocurre de forma transparente durante la asignación con el signo igual (=).

Gemini Notebook

# Entidad Rica vs Modelo Anémico

## Modelo Anémico




## Entidad Rica de Dominio

El objeto se protege y calcula sus propias reglas dinámicamente. (Clean Architecture).

Gemini Notebook

# Micro-reto Práctico en Clase

15:00 MIN

**Sistema:** Módulo de Inventario | **Clase:** StockArticulo

- ✅ Atributos: ‘sku’ (Público), ‘_existencias’ (Protegido).
- ✅ Lectura: Usar ‘@property’ para leer existencias.
- ✅ Seguridad: Usar ‘@existencias.setter’ para lanzar ‘ValueError’ si hay rotura de stock (negativo).
- ✅ Log: Implementar ‘__repr__’ con formato ‘[SKU] - N unidades’.

⚠ **PROHIBICIÓN ABSOLUTA:** Usar prefijos ‘get_’ o ‘set_’. ⚠

Gemini Notebook

# El Estándar de Oro

En Java, esto habría requerido el triple de líneas y clases accesorias.

```python
class StockArticulo:
    def __init__(self, sku: str, existencias: int):
        self.sku = sku
        self.existencias = existencias

    @property
    def existencias(self) -> int:
        return self._existencias

    @existencias.setter
    def existencias(self, valor: int):
        if valor < 0: raise ValueError('Rotura de stock')
        self._existencias = valor

    def __repr__(self) -> str:
        return f'[{self.sku}] - {self._existencias} unidades'
```

**Next Steps:** Semana 2

- Contratos Abstractos (abc.ABC / Protocol)

- Gestión Segura de Recursos (Context Managers)

Gemini Notebook